

Injaaz — Full Project Scope, Methods, and Techniques

This document describes what the Injaaz application covers (scope) and how it is built (methods, patterns, and libraries). It complements APPLICATION_OVERVIEW.md (high-level) and PROJECT_STRUCTURE.md (folders).

Last updated: April 2026 (align with repository; adjust when modules change).

1. Project scope (what the product does)

Injaaz is a facilities and operations web platform. In one deployable application it provides:

| Area | Scope |

|-----|-----|

| Identity & access | User accounts, passwords (hashed), JWT sessions, role and per-module access flags (HVAC, Civil, Cleaning, HR, Procurement, etc.). |

| Operational forms | Digital capture for HVAC/MEP, Civil, Cleaning, and cross-trade Inspection workflows—typically site visits, checklists, photos, signatures, and structured JSON form_data on submissions. |

| Workflow | Multi-stage review pipelines (supervisor, operations manager, BD, procurement, general manager—depending on module), pending queues, history, notifications. |

| HR | Many HR form types under /hr, employee → HR → General Manager approval, notifications, DOCX (templates + placeholders), PDF (native ReportLab layouts), HTML print views. |

| Procurement | Procurement-specific flows under /procurement. |

| MMR (reports) | CAFM-oriented analytics: Excel/HTML ingestion, pandas processing, chargeable rules, dashboards, Excel report output, scheduled email (APScheduler; default Asia/Dubai). |

| Business Development | BD routes under /bd and related admin entry. |

| DocHub | Document hub API under /api/docs and UI paths such as /dochub for authorized users. |

| Administration | User management, access control, device/admin tooling via /api/admin and related templates. |

| Reporting API | /api/reports for on-demand report regeneration where implemented. |

| Client experience | Server-rendered HTML/CSS/JS, PWA assets (manifest, service worker), optional Capacitor for mobile shells. |

The codebase is a modular monolith: many Flask blueprints, one database, shared app.models, common/, and config.py.

2. Architectural methods

2.1 Application factory and composition

- Injaaz.py defines create_app(): loads config, validates it, initializes Flask-SQLAlchemy, bcrypt, Flask-Migrate, JWTManager, registers blueprints, error handlers, and CLI hooks.

- Defensive imports: each major blueprint is imported in a try/except block so a broken optional module does not prevent the rest of the app from starting (routes for that module may be absent until fixed).

2.2 Configuration

- config.py: central place for paths (BASE_DIR, GENERATED_DIR, UPLOADS_DIR, JOBS_DIR), limits (MAX_CONTENT_LENGTH, per-file caps), environment-driven secrets and service URLs (DATABASE_URL, REDIS_URL, Cloudinary, mail, JWT lifetimes, cookie flags, MMR schedule overrides).
- common/config_validator.py: startup checks (e.g. weak secrets in production) so misconfiguration fails fast where implemented.

2.3 Data layer

- SQLAlchemy ORM in app/models.py: users, submissions, jobs, files, sessions, audit logs, notifications, module-specific tables as needed.
- Submission.form_data: JSON column holding flexible form payloads. Important technique: when updating nested dicts, assign a new dict (e.g. dict(submission.form_data or {})) before commit, so the ORM persists JSON changes reliably.
- Flask-Migrate for versioned schema changes; startup may also run db.create_all() and targeted ALTER TABLE for legacy DBs missing columns.
- SQLite in typical local dev; PostgreSQL required in production when FLASK_ENV is not development.

2.4 Authentication and authorization

- Flask-JWT-Extended: access + refresh tokens; identity is user id (string).
- Token locations: Authorization: Bearer and HTTP-only cookies (so browser navigation to some routes can work without JS setting headers).
- JWT cookie CSRF: default off in config comments so multipart uploads and cookie fallback do not break; can be enabled via env if all clients send CSRF headers.
- Session revocation: Session rows store JTI; token_in_blocklist_loader marks tokens revoked when the session is revoked; common/jwt_session.py can sync missing rows.
- Custom JWT error handlers: JSON 401 for /api/, redirects to login for HTML-oriented workflow pages where configured.
- Passwords: Flask-Bcrypt hashing (User.set_password / check_password).
- Authorization: mix of role (e.g. admin), designation (e.g. general_manager, hr_manager), and boolean flags (access_hr, access_civil, ...). Admins typically bypass module checks in helpers.

2.5 HTTP API style

- JSON APIs return structured bodies; many routes use common/error_responses.py (error_response, success_response) for consistent { success, error, error_code, details } shape.
- Flask-Limiter optional integration for rate limiting (e.g. login).
- Flask-WTF / CSRF where applicable on server-rendered forms; API routes often JWT-protected instead.

2.6 Files, media, and async-style work

- Uploads under GENERATED_DIR (configurable, e.g. persistent disk on Render).
- Cloudinary and boto3 (S3) in stack for cloud asset storage where configured.
- concurrent.futures.ThreadPoolExecutor in Injaaz.py for background report work (worker count tuned for small hosts).
- Redis + RQ available for queues/caching when REDIS_URL is set (rate limiting and workers).

2.7 Document and report generation (techniques)

| Technique | Libraries / approach | Typical use |

|-----|-----|-----|

| PDF (programmatic layout) | ReportLab | HR PDFs (module_hr/hr_pdf_builder.py), tables, typography, embedded signature images from base64 data URLs. |

| DOCX from templates | docxtpl, python-docx | HR Word templates with {{ placeholders }}; merged context from submission. |

| Excel read/write | openpyxl, XlsxWriter, pandas, xlrd | MMR and module reports; validation and analytics. |

| HTML in “Excel” exports | beautifulsoup4, lxml, html5lib | Parse CAFM exports that are HTML tables masquerading as spreadsheets. |

| Images in PDFs | Pillow | Signature cleanup / sizing where used. |

| Optional Windows DOCX→PDF | docx2pdf | Environment-dependent helpers where installed. |

2.8 Scheduling and email

- APScheduler for MMR (e.g. daily report email); timezone and hour/minute often driven by env (default Dubai).
- Email: SMTP-style config, Mailjet / Brevo-style API keys documented in config.py comments.

2.9 Frontend and mobile

- Jinja2 templates under templates/ and module templates/.
- Static assets: CSS, JS, PWA manifest and service worker, photo upload queue scripts.
- Capacitor (see repo / npm) for native wrappers; not all logic is duplicated—core is still the Flask app.

2.10 Testing and quality

- pytest with tests/conftest.py: in-memory SQLite, create_app(), JWT expiry disabled for tests, fixtures for users and auth headers.
- Scripts under scripts/ for HR PDF/DOCX smoke tests, DB init, template fixes—treat as regression helpers, not a full CI matrix.

3. Blueprint map (URL scope)

Registered from create_app() when imports succeed (prefixes as in code today):

| Blueprint | Typical prefix | Role |

|-----|-----|-----|

| hvac_mep_bp | /hvac-mep | HVAC & MEP module UI + APIs |

civil_bp	/civil	Civil module
cleaning_bp	/cleaning	Cleaning module
auth_bp	/api/auth	Login, register, JWT issue/refresh
admin_bp	/api/admin	Admin APIs
workflow_bp	/api/workflow	Workflow queues and related APIs
bd_bp	/bd	Business development
docs_bp	/api/docs	DocHub API
hr_bp	/hr	HR UI + /hr/api/
procurement_module_bp	/procurement	Procurement
inspection_bp	/inspection	Inspection module
mmr_bp	/admin/mmr	MMR upload, dashboard, reports, schedule
reports_bp	/api/reports	Reports API

4. Important directories (technique → location)

Location	Contents
app/	Core models, auth, admin, workflow helpers, reports API, services, tasks
common/	Config validation, JWT session sync, security, utils, retries, shared validation
module_	Domain blueprints: routes, templates, generators, module-specific JSON/data
templates/, static/	Shared UI and assets
migrations/	Alembic / Flask-Migrate versions
scripts/	Operational and test scripts
docs/	Architecture and setup documentation
generated/ (or GENERATED_DIR)	Uploads, jobs, ephemeral outputs (configure for production persistence)

5. Security and operations (summary)

- Secrets and DB URLs via environment variables; never commit .env.
- HTTPS-only cookies in production when SESSION_COOKIE_SECURE / JWT cookie secure flags are set appropriately.
- Default admin password must be changed on first deploy; app may log generated password if env not set—treat as emergency-only.
- CORS and security headers: follow deployment guides (CLOUD_ONLY_SETUP.md, Render runbooks).
- Dependency pinning in requirements-prods.txt; periodic upgrades with regression testing.

6. How to keep this document useful

- When adding a new blueprint, extend Section 3 and APPLICATION_OVERVIEW.md.

- When adding a new library, extend Section 2.7 and requirements-prods.txt comments if non-obvious.
- When changing auth or JSON update patterns, update Sections 2.4 and 2.3 so future developers avoid subtle bugs.

7. Related documents

| Document | Use |

|-----|-----|

| APPLICATION_OVERVIEW.md | Shorter stakeholder overview |

| ../README.md | Quick start and tech stack one-pager |

| ../SETUP.md | Environment and troubleshooting |

| ../PROJECT_STRUCTURE.md | Directory tree |

| ../PROJECT_FLOW.md | User/workflow narrative |

| module_mmr/CHARGEABLE_RULES.md | MMR business rules |

| docs/Injaaz_Project_Overview.pdf | Generated PDF summary (run python scripts/generate_project_overview_pdf.py) |

This file is the canonical “scope + methods + techniques” reference for the Injaaz codebase.